

Indian Institute of Technology Jammu
CT1 — Computer Systems and Programming (CS-1-01 MO)
WORKED SOLUTIONS

Time: 1 Hour	Maximum Marks: 50
Name: _____	Roll No.: _____

Instructions:

1. Attempt as many questions as you like — there is no penalty for extra attempts.
2. Section A: your **best 5 of 7** count (2 marks each).
3. Section B: your **best 5 of 7** count (4 marks each).
4. Section C: your **best 2 of 3** count (10 marks each).
5. Show your working. Answers without steps earn partial credit at best.

How to use this document. Every question below carries three things:

Answer. the result on its own, so you can check yours quickly.

Working. every step, written out — the conversions, the carries, the traces. Where the paper says “showing every step”, these steps *are* the marks; an answer arrived at silently cannot score full credit even when it is right.

Where this goes wrong. the specific mistakes this question invites, and how to catch each one yourself.

Read the last of the three even where you got the question right — most of the marks lost on this paper went to a small number of recurring slips, and nearly all of them are caught by a check that takes a few seconds: convert your answer back, add your negative to its positive and confirm you get zero, or trace one line at a time instead of computing in your head. Work the question again from a blank page before reading on; a solution you follow is not the same as one you can produce.

SECTION A — Multiple Choice Questions

Best 5 of 7. $5 \times 2 = 10$ Marks

1. Which decimal number is $(14)_{16}$?
A. 14 B. 18 C. 20 D. 41

Answer. C — 20.

Working. A numeral is a sum of digit $\times$ place value. In base 16 the places are $\dots, 16^2, 16^1, 16^0$, so

$$(14)_{16} = 1 \times 16^1 + 4 \times 16^0 = 16 + 4 = 20.$$

Where this goes wrong. Option A is there for exactly one reason: the digits look like “fourteen”. They are not. The same two marks on paper mean 14 in base 10, 20 in base 16, 12 in base 8 and 9 in base 5 — *a numeral means nothing until you are told the base*. Whenever a question writes a subscript, read it first and convert before doing anything else.

2. In 4-bit **one’s complement**, how many distinct bit patterns represent the value zero?
A. 0 B. 1 C. 2 D. 16

Answer. C — two.

Working. In one’s complement a negative is formed by flipping every bit of the positive. Apply that to zero: $+0 = 0000$, and flipping every bit gives 1111 , which is therefore -0 . Both patterns denote the value zero, so there are two of them.

Where this goes wrong. Answering B means reasoning about the *value* (“zero is one number”) when the question asks about *patterns*. Those are different questions. Note also that this is not a quirk of four bits: flip the all-zeros pattern at any width and you get all-ones, so one’s complement has two zeros at every width. That duplicate is the flaw two’s complement was designed to remove — which is why it is worth remembering rather than deriving.

3. What is the chief advantage of **two’s complement** over sign-magnitude and one’s complement?
- A. It can represent more positive numbers
 - B. Zero has exactly one pattern, and one ordinary binary adder handles all signs
 - C. Negative numbers need fewer bits
 - D. Overflow can never happen

Answer. B — one pattern for zero, and one ordinary adder handles every sign.

Working. Two’s complement is chosen for what it does to the *hardware*. Because $a - b$ can be computed as $a +$ (two’s complement of b), the machine needs no separate subtractor and no test of the operands’ signs: the same adder runs every case. And the duplicate zero of one’s complement is gone, so “is this value zero?” is a single comparison.

Where this goes wrong. **A** is the tempting wrong answer, and it is worth being precise about why. A k -bit two’s-complement machine holds $-2^{k-1} \dots 2^{k-1} - 1$: it gains one extra *negative* value, not a positive one. In 4 bits, sign-magnitude reaches $+7$ and two’s complement also reaches $+7$ — the difference is at the bottom (-8 against -7). **D** is false outright: overflow is a consequence of finite width and no encoding escapes it.

4. A 4-bit two’s-complement machine computes $5 + 6$. The stored result represents:
- A. 11 B. -5 C. -6 D. 1

Answer. B — it represents -5 .

Working. Add the two patterns as ordinary binary:

$$\begin{array}{r} 0101 = 5 \\ + 0110 = 6 \\ \hline 1011 \end{array}$$

A 4-bit two’s-complement machine can hold only $-8 \dots 7$, and 11 is outside it. To read the stored pattern, subtract 2^4 because the leading bit is 1: $11 - 16 = -5$.

Where this goes wrong. Two errors both lead away from B. The first is answering **A**: the arithmetic $5 + 6 = 11$ is right, but the question asks what the *machine stores*, and 11 has nowhere to live. The second is trusting the result because nothing visibly failed — there is no error message, no crash, just a wrong number. The tell is the sign bit: two positives were added and the result’s sign bit is 1. **Adding two positives can never legitimately give a negative**, so that alone proves overflow without converting anything back.

5. You run `gcc hello.c -o hello`. What is inside the new file `hello` that was *not* inside `hello.c`?
- A. A neater, error-checked copy of the C text
 - B. Machine instructions the processor can execute directly
 - C. The output “Hello World”, computed in advance
 - D. Nothing new — it is the same file under another name

Answer. B — machine instructions the processor can execute directly.

Working. *gcc translates.* The C file holds text that a human wrote and a processor cannot obey; the output file holds instruction codes in the processor’s own repertoire. Running `./hello` hands those codes to the processor, which is the first moment anything executes.

Where this goes wrong. C is the instructive wrong answer. Compiling and running are two separate events: at the end of `gcc` nothing has run, so the text “Hello World” cannot yet have been produced — it appears only when you run the program, and it appears again every time you run it. If the output really were computed in advance and stored, a program that asks for your name could not exist. **D** is contradicted by the file itself: compare the sizes, or open both.

6. What exactly does this program print?

```
#include <stdio.h>
int main(void) {
    printf("Hello");
    printf(" World\n");
    printf("CS-1-01");
    return 0;
}
```

- A. Hello, World, CS-1-01 on three separate lines
- B. Hello World on one line, then CS-1-01 on the next
- C. Hello World CS-1-01 all on one line
- D. HelloWorld\nCS-1-01 printed literally

Answer. B — Hello World on one line, then CS-1-01 on the next.

Working. `printf` prints exactly the characters it is given and nothing more. Follow the output position as each call runs:

<code>printf("Hello");</code>	the line so far reads <code>Hello</code>
<code>printf(" World\n");</code>	appends <code>World</code> , then the <code>\n</code> ends the line
<code>printf("CS-1-01");</code>	starts the next line with <code>CS-1-01</code>

So the newline comes from the one `\n` in the middle string — not from the fact that there are three separate calls.

Where this goes wrong. A assumes each `printf` ends a line. It does not: a separate call is not a separate line. **D** assumes `\n` is printed literally as a backslash and an `n`; inside a string it is one character, the newline. The reliable habit is to read a `printf` as “append exactly these characters”, and ask where the newlines are rather than how many calls there are.

7. According to this course, programming is ultimately about:

- A. memorizing the syntax of C correctly
- B. making the computer run as fast as possible
- C. solving a problem — turning a precise plan into instructions an executor can carry out, then checking the result
- D. writing as many lines of code as possible

Answer. C — solving a problem: a precise plan, carried out, then checked.

Working. This is the course’s own claim, and everything in it has argued for it: the Turtle had no C in sight, the Toy Computer had four instructions, and the same add-two-numbers plan survived unchanged across all of them. The computer is the instrument; the plan and its verification are the subject.

Where this goes wrong. **A** and **B** are each a real skill mistaken for the goal. Syntax is worth knowing but it is the notation, not the idea — and a syntactically perfect program can be entirely wrong. Speed matters once a program is *correct*; a fast wrong answer is worthless. **D** inverts the truth: a shorter program that solves the problem is better, not worse.

SECTION B — Short Answer Questions

Best 5 of 7. $5 \times 4 = 20$ Marks

1. Two conversions, showing every step:

(a) Convert $(11001)_2$ to decimal, writing out the powers of 2 you used.

(b) Convert $(25)_{10}$ to hexadecimal, using repeated division by 16.

Answer. (a) 25. (b) $(19)_{16}$.

Working. (a) Write the place value above each bit, then add the ones that are set:

bit	1	1	0	0	1
place value	$2^4 = 16$	$2^3 = 8$	$2^2 = 4$	$2^1 = 2$	$2^0 = 1$
contributes	16	8	—	—	1

$$16 + 8 + 1 = 25.$$

(b) Divide by 16 repeatedly; the remainders are the hex digits, *read upwards*:

$25 \div 16$	quotient 1	remainder 9	(least significant digit)
$1 \div 16$	quotient 0	remainder 1	(most significant digit)

Reading the remainders bottom-to-top gives $(19)_{16}$. Check by going back: $1 \times 16 + 9 = 25$. ✓

Where this goes wrong. Three things cost marks here. First, **reading the remainders downwards** gives $(91)_{16} = 145$ — always check by converting back, which takes five seconds. Second, writing the answer as plain “19” with no base: in this question the whole point is that $(19)_{16}$ and $(19)_{10}$ are different numbers, so the subscript is part of the answer. Third, part (a) and part (b) are the *same* twenty-five written two ways; if your two answers cannot be reconciled, one of them is wrong.

2. Write -5 in 4 bits under each convention, showing your steps:

(a) sign-magnitude

(b) one’s complement

(c) two’s complement

Answer. (a) 1101 (b) 1010 (c) 1011.

Working. Every convention starts from the same place: $+5 = 0101$.

(a) sign-magnitude	keep the magnitude 101, set the sign bit to 1	1101
(b) one’s complement	flip every bit of 0101	1010
(c) two’s complement	flip every bit, then add 1: $1010 + 1$	1011

Check (c) by reading the pattern back: 1011 has its leading bit set, so its value is $11 - 16 = -5$.
✓

Where this goes wrong. The usual loss is **giving the same pattern for two of the three**, which cannot be right — the question asks for three different conventions precisely because they disagree. Two specific slips: flipping the magnitude bits but forgetting the +1 in (c), which silently answers (b) again; and flipping *all four* bits in (a), when sign-magnitude leaves the magnitude alone and only sets the sign bit. A free check: in two's complement, $0101 + 1011$ must give 0000 (discarding the carry), because $5 + (-5) = 0$.

3. On a 4-bit **one's complement** machine, add (+3) and (−3) by binary addition, showing every bit. What pattern do you obtain, what value does it represent, and what does this tell you about why two's complement was preferred?

Answer. The pattern is 1111, which represents −0 — a second zero.

Working. In one's complement −3 is +3 with every bit flipped: $0011 \rightarrow 1100$. Add:

$$\begin{array}{rcl} & 0011 & = +3 \\ + & 1100 & = -3 \\ \hline & 1111 & = -0 \end{array}$$

The sum is arithmetically correct — it *is* zero — but it is written in the second of the two available zero patterns, not 0000.

Where this goes wrong. Two traps. The first is expecting 0000 and assuming you have made an arithmetic error when 1111 appears; the addition is right, and the oddity is the *encoding*, which is the entire point of the question. The second is stopping at the pattern without answering the third part — the question asks what this tells you about two's complement, and that sentence carries marks. It is this: a machine that must treat 0000 and 1111 as equal needs a special case in every comparison, whereas two's complement has exactly one zero and needs none.

4. On a 4-bit two's-complement machine, compute $3 - 5$ the way the hardware does: form the two's complement of 5, add it to 3 showing every bit (discard any carry out of the fourth bit), and interpret the resulting pattern. What circuit did the machine just avoid needing?

Answer. 1110, which is −2. The machine avoided needing a subtractor circuit.

Working. Form the two's complement of 5: flip 0101 to 1010, add 1 to get 1011. Then *add* it to 3:

$$\begin{array}{rcl} & 0011 & = +3 \\ + & 1011 & = -5 \\ \hline & 1110 & \end{array}$$

The leading bit is 1, so the value is $14 - 16 = -2$. And indeed $3 - 5 = -2$. ✓

Where this goes wrong. The commonest loss is doing the subtraction *as* a subtraction — borrowing across the bits — and getting −2 by a route the question did not ask for. The answer is then right and the marks are not, because the question is about the mechanism: subtraction performed by the *addition* circuit. The second slip is forgetting the +1 when complementing, which gives $0011 + 1010 = 1101 = -3$: wrong by exactly one, the signature of that mistake.

5. (a) Take the 4-bit pattern $d = 0110$. Compute the one's complement of its one's complement, showing both steps. State what you observe, and argue in one sentence why it must hold for *every* pattern. [2]

(b) For a k -bit machine, the two's complement of d can be computed as the unsigned number $2^k - d$. Write the analogous formula for the *one's* complement, and verify your formula for $d = 5$, $k = 4$ against bit-flipping. [2]

Answer. (a) 0110 — you get back the pattern you started with. (b) $(2^k - 1) - d$.

Working. (a) Flip, then flip again:

$$0110 \longrightarrow 1001 \longrightarrow 0110.$$

It must hold for every pattern, because the operation acts on each bit independently and a single bit flipped twice is itself: $0 \rightarrow 1 \rightarrow 0$ and $1 \rightarrow 0 \rightarrow 1$. An operation that undoes itself is called an *involution*. (b) Flipping every bit of d is the same as subtracting d from the all-ones pattern, and the all-ones pattern of width k is $2^k - 1$. So the one's complement is $(2^k - 1) - d$. Verify at $d = 5$, $k = 4$: the formula gives $15 - 5 = 10 = 1010$, and flipping 0101 by hand also gives 1010. ✓

Where this goes wrong. In (a), the observation is only half the answer — the marks are for saying *why* it must hold for every pattern, and “each bit is flipped twice” is that reason. In (b) the frequent answer is $2^k - d$, which is the *two's* complement; the one's complement is one less. If you cannot recall which, derive it: the two differ by exactly the +1 in the definition, so their formulas must differ by 1 too. And do the verification the question asks for — it is worth marks on its own, and it catches the off-by-one immediately.

6. Mark each statement **True or False** and justify with a concrete example:

(a) “Every real number can be represented exactly on a computer.”

(b) “No real number can be represented exactly on a computer.”

Answer. (a) **False.** (b) **Also false.** Both statements are wrong.

Working. (a) A k -bit format has at most 2^k distinct patterns, so it can name at most 2^k values — finitely many. The real numbers are infinite, so almost all of them have no exact pattern. Concretely, 0.1 in binary is $0.00011\overline{1}$, a non-terminating fraction, so it is stored *rounded*: this is why $0.1 + 0.2$ does not print as 0.3. (b) But it does not follow that nothing is exact. Any value of the form $m \times 2^e$ that fits the format is stored exactly — for instance $0.5 = 2^{-1}$, or $6.5 = 1.101_2 \times 2^2$. Both are exact in IEEE format, with no rounding whatsoever.

Where this goes wrong. This question is built so that the two halves *look* contradictory, and the mistake is to assume one of them must be true and hunt for which. They are both false, and the truth lies between them: *some* reals are exact, most are not. Two ways marks were lost: giving the verdicts without an example (the question says “justify with a concrete example”, so each verdict needs one), and answering (b) with an argument about *precision* — “it is very close” — when the claim is about exactness, and 0.5 settles it.

7. A famous line — which the internet confidently attributes to Edsger Dijkstra — says that computer science is about computers as much as astronomy is about telescopes.

(a) What is the analogy claiming? If programming is not “about” the computer, what *is* it about? Support your answer with one concrete example from this course (the Turtle, the Toy Computer, or your first C program). [2]

(b) The attribution above is, in fact, wrong — and that is itself a lesson this course keeps repeating. In one sentence: what should you do with a claim that is plausible and widely repeated? [2]

Answer. (a) That the computer is the instrument, not the subject; the subject is the *plan*. (b) Check it against a source before repeating it.

Working. (a) Astronomy uses telescopes but studies the sky; the telescope is apparatus. In the same way computing uses computers but studies *plans made precise enough to be executed and checked* — algorithms. Any one of the course’s examples supports it: the Turtle had no silicon in sight and everything that mattered was the plan; the Toy Computer obeyed four instructions and still made the point about order; and the add-two-numbers plan survived unchanged from toy words into C, because the plan was never about the machine in the first place. (b) Verify before repeating. Plausible, quotable and widely attributed is not the same as true. This line is almost always credited to Dijkstra; the attribution traces instead to Michael Fellows, with Ian Parberry, in the early 1990s.

Where this goes wrong. Part (a) asks for an example and the marks are split accordingly — an answer that explains the analogy well but names nothing from the course cannot get full marks. Part (b) is not a trick: the question tells you the attribution is wrong, so no credit turns on knowing who really said it. The marks are for the *habit* it is asking about, stated as a rule you would actually follow. Any answer amounting to “check the source before believing or repeating it” earns them.

SECTION C — Long Answer Questions

Best 2 of 3. $2 \times 10 = 20$ Marks

1. All in 8-bit two’s complement, showing every carry:

(a) Compute $93 + 45$ in binary. State the decimal value the stored pattern represents, whether overflow occurred, and *how you can tell without converting back*. [5]

(b) Compute $93 - 45$ as an addition (using the two’s complement of 45), showing the steps and what happens to the ninth bit. [5]

Answer. (a) 10001010, which represents -118 ; **overflow occurred**. (b) 00110000 = 48, the ninth bit falling off the edge.

Working. (a) $93 = 01011101$ and $45 = 00101101$. Column by column, from bit 0 (rightmost):

bit	93	45	carry in	sum	carry out
7	0	0	1	1	0
6	1	0	1	0	1
5	0	1	1	0	1
4	1	0	1	0	1
3	1	1	1	1	1
2	1	1	0	0	1
1	0	0	1	1	0
0	1	1	0	0	1

The stored pattern is 10001010. Its leading bit is 1, so its value is $138 - 256 = -118$.

How to tell overflow without converting back — two independent tests, and they always agree:

The sign test. Two positive numbers were added and the sign bit of the result is 1. A positive plus a positive cannot be negative, so the result is not the true sum.

The carry test. The carry *into* the sign bit is 1 but the carry *out* of it is 0. When those two differ, the result has overflowed.

The true sum is 138, and an 8-bit two’s-complement machine holds only $-128 \dots 127$.

(b) Now $93 - 45$, done as an addition. First -45 : flip 00101101 to 11010010, add 1, giving 11010011.

bit	93	-45	carry in	sum	carry out
7	0	1	1	0	1
6	1	1	0	0	1
5	0	0	1	1	0
4	1	1	1	1	1
3	1	0	1	0	1
2	1	0	1	0	1
1	0	1	1	0	1
0	1	1	0	0	1

The carry out of bit 7 is 1 — this is the “ninth bit”. It falls off the 8-bit edge and is discarded, leaving $00110000 = 48$. And $93 - 45 = 48$. ✓ Here the carry into the sign bit is 1 and the carry out is also 1: they agree, so there is *no* overflow, and the discarded bit is harmless.

Where this goes wrong. The most expensive mistake is in (b): seeing a carry out of the top bit and calling it overflow. It is not. A carry out on its own means nothing — in (b) one appears and the answer is perfectly correct. Overflow is when the carry *in* to the sign bit and the carry *out* of it **differ**, which is what happened in (a) and not in (b). Reading part (a)’s pattern as 138 also loses marks: the question asks what the stored pattern *represents*, and on a two’s-complement machine a leading 1 means a negative value. Finally, (a) asks three things — the value, whether overflow occurred, and how you can tell without converting — and that third part is worth marks on its own.

2. For a k -bit two’s-complement machine:

(a) What is the **largest positive** integer it can represent, and what is the **most negative**? Justify each in a sentence or two — think about how the sign bit splits the 2^k available patterns — and explain why the two ends are not symmetric. [4]

(b) Now let the machine test your claim. On a **4-bit** machine your answer says the largest is $7 = 0111$. Add 1 to it in binary. What pattern appears, what value does it represent, and does the result support or contradict your claim in (a)? What is this phenomenon called? [3]

(c) Repeat the test at the other end: take the most negative value and add $-1 = 1111$, showing what happens to the carry. What do you observe, and what is *this* called? [3]

Answer. (a) Largest $2^{k-1} - 1$, most negative -2^{k-1} . (b) $1000 = -8$; it *supports* the claim. Overflow. (c) $0111 = +7$. Underflow (equally: negative overflow).

Working. (a) There are 2^k patterns. Those with the sign bit set — exactly half, 2^{k-1} of them — are the negatives, so the most negative is -2^{k-1} , written 1 followed by all zeros. The other half must accommodate the positives *and* zero. Zero consumes one of them, so only $2^{k-1} - 1$ are left for the positives, and the largest is $2^{k-1} - 1$, written 0 followed by all ones.

Why the ends are not symmetric: zero is neither positive nor negative but still needs a pattern, and it is stored on the non-negative side. That one pattern is the entire asymmetry. At $k = 4$: $-8 \dots +7$.

(b) Add 1 to the largest, 0111:

bit	7	1	carry in	sum	carry out
3	0	0	1	1	0
2	1	0	1	0	1
1	1	0	1	0	1
0	1	1	0	0	1

The result is $1000 = -8$: one past the largest positive lands on the most negative. This **supports** the claim in (a) — there was nowhere higher to go, so the count wrapped. The carry into the sign bit is 1 and out of it is 0: they differ, so this is **overflow**.

(c) Now the other end: the most negative, 1000, plus $-1 = 1111$:

bit	-8	-1	carry in	sum	carry out
3	1	1	0	0	1
2	0	1	0	1	0
1	0	1	0	1	0
0	0	1	0	1	0

The carry out of bit 3 is 1; it falls off the 4-bit edge and is discarded, leaving $0111 = +7$. One *below* the most negative wraps round to the largest positive. Here the carry in to the sign bit is 0 and the carry out is 1 — they differ again, so this too is a genuine overflow. Between them, (b) and (c) locate both ends of the range exactly where (a) predicted.

Where this goes wrong. Part (b) asks whether the result supports or contradicts your claim, and the instinct is to say “contradicts” — the machine seems to have disproved that 7 was the largest. It has not: it produced a *wrong* answer, which is precisely what running out of range looks like. In (c), discarding the fifth bit is not optional bookkeeping; keep it and you get 10111, which is not a 4-bit value at all. On naming (c): most architecture texts call this **negative overflow** and reserve “underflow” for floating-point values too small to represent. Either term earned full marks here.

3. A toy machine has a single box for a number, and it obeys — blindly, one line at a time, top to bottom — exactly these four instructions:

```
LOAD n    put the number n into the box
ADD n     add n to what the box holds
DOUBLE    double what the box holds
PRINT     print the box's value on its own line
```

Consider this program:

```
LOAD 3
ADD 2
DOUBLE
PRINT
ADD 1
PRINT
```

- (a) Predict the exact output, tracing the box’s value step by step. [3]
 (b) The lines ADD 2 and DOUBLE are now swapped. Predict the new output. [3]
 (c) The machine did not change between (a) and (b), yet the output did. In one sentence: where does the “meaning” of a program live? [2]
 (d) Using only these four instructions, write a program that prints exactly 7 and then 14, on two lines. [2]

Answer. (a) 10 then 11. (b) 8 then 9. (c) In the plan — the order of the instructions. (d) e.g. LOAD 7 / PRINT / DOUBLE / PRINT.

Working. Trace the box one line at a time. Write the value *after* each instruction runs.

(a) as given			(b) two lines swapped		
instruction	box	printed	instruction	box	printed
LOAD 3	3		LOAD 3	3	
ADD 2	5		DOUBLE	6	
DOUBLE	10		ADD 2	8	
PRINT	10	10	PRINT	8	8
ADD 1	11		ADD 1	9	
PRINT	11	11	PRINT	9	9

(c) The machine is identical in both columns; only the order changed, and the output changed with it. So the meaning lives in the *plan*, not in the machine — which obeys one line at a time and understands nothing.

(d) `LOAD 7` puts 7 in the box, `PRINT` shows it, `DOUBLE` makes it 14, `PRINT` shows that. Any program that prints exactly 7 then 14 earns full marks — `LOAD 0 / ADD 7 / PRINT / ADD 7 / PRINT` is just as correct. Shorter is more elegant, not more right.

Where this goes wrong. The single biggest loss on this question is **doing the arithmetic in your head instead of tracing**. In (b), 3 doubled is 6 and then +2 is 8 — but read quickly, “double and add 2” invites $3 + 2 = 5$ then doubled = 10, which is (a)’s answer given for (b). Writing the box’s value on its own line after each instruction costs fifteen seconds and makes the mistake impossible; it is also exactly what part (a) asks for (“tracing the box’s value step by step”), so the trace itself carries marks.

In (c), an answer about the machine (“it just follows instructions”) describes what happened but does not answer where the meaning *lives*. The marks are for identifying the plan — specifically its order — as the thing that determines the result.

In (d), check your program against the spec before moving on: it must print 7 and 14 *on two lines*, so it needs two `PRINT`s. A program printing only 14 is a misread, not a miscalculation.

— End of Question Paper —